Totpal - Netflix Viewership Code

September 21, 2025

Title: Weeks 1 & 2 Assignment - Netflix Viewership

Author: Allyson Totpal

Date: 21 September 2025

Modified By: Allyson Totpal

Description: Project on Non-English Netflix viewership and it's growth in popularity. Report is written at bottom of notebook.

[3]:
```
# import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import matplotlib.dates as mdates
import warnings
warnings.simplefilter("ignore")
```

[5]:
```
# read in data
df_pop = pd.read_excel("/Users/smooshii/DSC640/most-popular-netflix.xlsx")
df_countries = pd.read_excel("/Users/smooshii/DSC640/
 ↪all-weeks-countries-netflix.xlsx")
df_global = pd.read_excel("/Users/smooshii/DSC640/all-weeks-global-netflix.
 ↪xlsx")
```

[7]:
```
df_pop.head()
```

[7]:
```
          category  rank              show_title season_title  \
0  Films (English)     1              Red Notice          NaN
1  Films (English)     2           Don't Look Up          NaN
2  Films (English)     3        The Adam Project          NaN
3  Films (English)     4                Bird Box          NaN
4  Films (English)     5  Leave the World Behind          NaN

   hours_viewed_first_91_days  runtime  views_first_91_days
0                   454200000   1.9667            230900000
1                   408600000   2.3833            171400000
```

```
2                    281000000   1.7833            157600000
3                    325300000   2.0667            157400000
4                    339300000   2.3667            143400000
```

[9]:
```
df_countries.head()
```

[9]:
```
  country_name country_iso2        week category  weekly_rank  \
0    Argentina           AR  2024-04-14    Films            1
1    Argentina           AR  2024-04-14    Films            2
2    Argentina           AR  2024-04-14    Films            3
3    Argentina           AR  2024-04-14    Films            4
4    Argentina           AR  2024-04-14    Films            5

                      show_title season_title  cumulative_weeks_in_top_10
0                  The Tearsmith          NaN                           2
1                         Stolen          NaN                           1
2                  Love, Divided          NaN                           1
3  Woody Woodpecker Goes to Camp          NaN                           1
4                  Rest In Peace          NaN                           3
```

[11]:
```
df_global.head()
```

[11]:
```
         week         category  weekly_rank                     show_title  \
0  2024-04-14  Films (English)            1              What Jennifer Did
1  2024-04-14  Films (English)            2  Woody Woodpecker Goes to Camp
2  2024-04-14  Films (English)            3                          Scoop
3  2024-04-14  Films (English)            4                          Glass
4  2024-04-14  Films (English)            5                    Megan Leavey

  season_title  weekly_hours_viewed  runtime  weekly_views  \
0          NaN             26100000   1.4500    18000000.0
1          NaN             19600000   1.6667    11800000.0
2          NaN             14600000   1.7167     8500000.0
3          NaN             11000000   2.1500     5100000.0
4          NaN              9700000   1.9333     5000000.0

   cumulative_weeks_in_top_10  is_staggered_launch episode_launch_details
0                           1                False                    NaN
1                           1                False                    NaN
2                           2                False                    NaN
3                           2                False                    NaN
4                           1                False                    NaN
```

[13]:
```
# coerce week into datetime
for df in [df_countries, df_global]:
    if "week" in df.columns:
        df["week"] = pd.to_datetime(df["week"], errors="coerce")
```

[15]:
```
# coerce numeric columns
num_cols_pop = ["rank", "hours_viewed_first_91_days", "runtime",␣
↪"views_first_91_days"]
for c in num_cols_pop:
    if c in df_pop.columns:
        df_pop[c] = pd.to_numeric(df_pop[c], errors="coerce")

num_cols_countries = ["weekly_rank", "cumulative_weeks_in_top_10"]
for c in num_cols_countries:
    if c in df_countries.columns:
        df_countries[c] = pd.to_numeric(df_countries[c], errors="coerce")

num_cols_global = ["weekly rank", "weekly_hours_viewed", "runtime",␣
↪"weekly_views", "cumulative_weeks_in_top_10"]
for c in num_cols_global:
    if c in df_global.columns:
        df_global[c] = pd.to_numeric(df_global[c], errors="coerce")
```

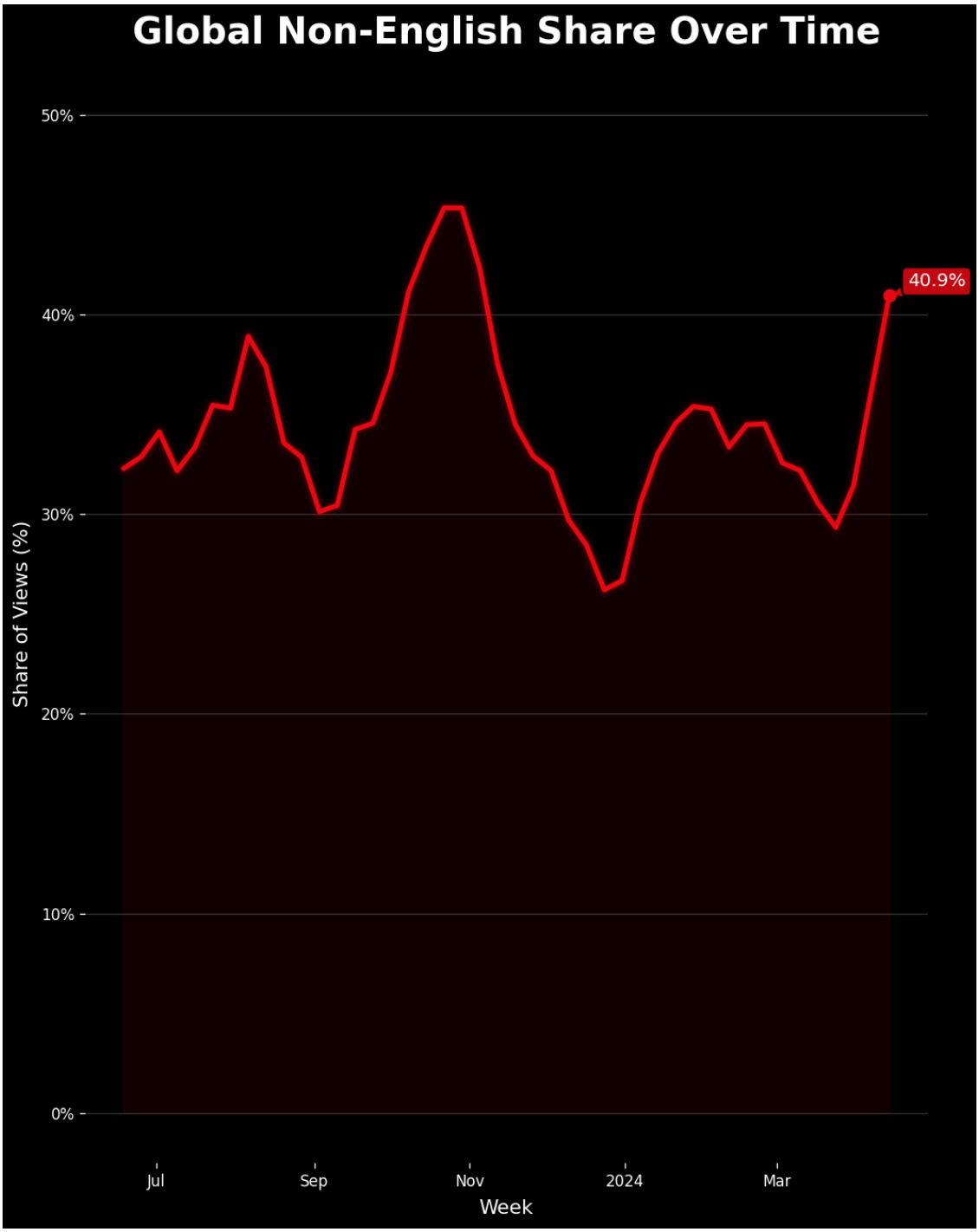
[17]:
```
# drop duplicates
df_pop = df_pop.drop_duplicates()
df_countries = df_countries.drop_duplicates()
df_global = df_global.drop_duplicates()
```

[19]:
```
# define metrics & language bucket

# prefer 'weekly_views' if it exists and has nonzero total
# otherwise, fall back to 'weekly_hours_viewed'
metric = "weekly_views" if ("weekly_views" in df_global.columns and
                            pd.to_numeric(df_global["weekly_views"],␣
↪errors="coerce").fillna(0).sum() > 0) else "weekly_hours_viewed"

def lang_bucket(cat):
    """map anything containing 'non-english' or 'non english' to 'Non-English';␣
↪else 'English'"""
    s = "" if pd.isna(cat) else str(cat).lower()
    return "Non-English" if "non-english" in s or "non english" in s else␣
↪"English"

# apply language bucket to global
df_global["lang_bucket"] = df_global["category"].apply(lang_bucket)
# boolean flag on countries indicating non-english
df_countries["is_ne"] = df_countries["category"].astype(str).str.contains("non-?
↪english", case = False, na = False)
```

0.0.1 Visual 1

[22]:
```
# global non-english share over time

# calculate share of non-english content by week
gc = df_global.dropna(subset=["week"]).copy()
tot = gc.groupby("week")[metric].sum() # total views per week
ne = gc[gc["lang_bucket"] == "Non-English"].groupby("week")[metric].sum() #␣
↪non-english views
share = (ne.reindex(tot.index, fill_value=0) / tot.replace(0, np.nan)).fillna(0)

# trim leading weeks with 0 share
first_nonzero = share[share > 0].index.min()
if pd.notna(first_nonzero):
    share = share.loc[first_nonzero:]

# smooth with 4-week rolling average for a cleaner curvve
share_smooth = share.rolling(window=4, min_periods=1).mean()

# set-up figure aesthetics
plt.style.use("dark_background")
fig = plt.figure(figsize=(9, 11.25), dpi=120)
ax = plt.gca()
ax.grid(True, which="major", axis="y", alpha=0.2, linewidth=0.8)
ax.grid(False, axis="x")
accent = "#E50914"
accent_dim = "#FF6B73"

# plot line with glow affect
for lw, alpha in [(10, 0.08), (6, 0.12), (4, 0.18)]:
    ax.plot(share_smooth.index, share_smooth.values * 100,
            linewidth=lw, color=accent, alpha=alpha, solid_capstyle="round")

# main trend line
ax.plot(share_smooth.index, share_smooth.values * 100,
        linewidth = 3.2, color = accent, solid_capstyle = "round")

# titles and axes
ax.fill_between(share_smooth.index, 0, share_smooth.values * 100,
                color=accent, alpha=0.08)
ax.set_title("Global Non-English Share Over Time", fontsize=24, pad=14,␣
↪weight='bold')
ax.set_xlabel("Week", fontsize=13, labelpad=6)
ax.set_ylabel("Share of Views (%)", fontsize=13, labelpad=6)
ax.yaxis.set_major_formatter(mtick.PercentFormatter(xmax=100))

# compact date labels
```

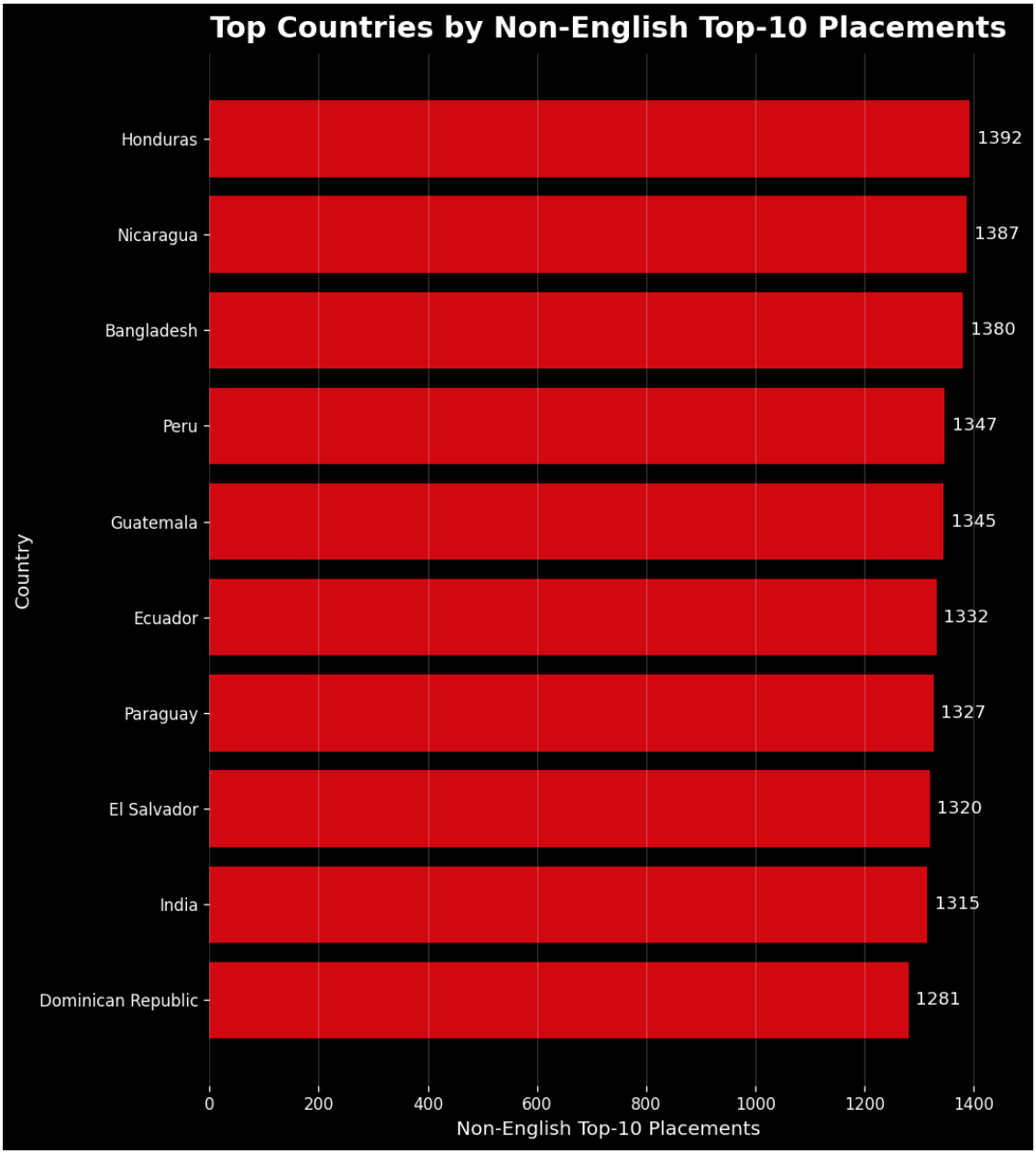
```python
try:
    ax.xaxis.set_major_locator(mdates.AutoDateLocator(minticks=4, maxticks=8))
    ax.xaxis.set_major_formatter(mdates.ConciseDateFormatter(ax.xaxis.
 ↪get_major_locator()))
except Exception:
    pass

# remove spines & faint reference line
for spine in ["top", "right", "left", "bottom"]:
    ax.spines[spine].set_visible(False)
ax.axhline(50, color="white", alpha=0.06, linewidth=1.0)

# last-point callout
last_x = share_smooth.index[-1]
last_y = float(share_smooth.iloc[-1] * 100)
label_txt = f"{last_y:.1f}%"

ax.scatter([last_x], [last_y], s=60, color=accent, zorder=3)
ax.annotate(
    label_txt,
    xy=(last_x, last_y),
    xytext=(12, 6), textcoords="offset points",
    fontsize=12, color="white",
    bbox=dict(boxstyle="round,pad=0.25", fc=accent, ec="none", alpha=0.85),
    arrowprops=dict(arrowstyle='-|>', color=accent, lw=1, alpha=0.8)
)

plt.tight_layout()
out_path = "visual1.png"
plt.savefig(out_path, bbox_inches="tight")
plt.show()
```


Global Non-English Share Over Time
50%
40%
30%
20%
10%
0%
Share of Views (%)
40.9%
Jul
Sep
Nov
2024
Mar
Week

0.0.2 Visual 2

[25]:
```
# top countries by top 10 non-english placements
cc = df_countries.copy()

# aggregate: ne top 10 placements per country
top_ne = (
    cc.query("weekly_rank >= 1 and weekly_rank <= 10 and is_ne == True")
      .groupby("country_name", as_index=False)["weekly_rank"]
      .count()
      .rename(columns={"weekly_rank": "ne_top10_entries"})
      .sort_values("ne_top10_entries", ascending=False)
)

# if no rows, infer ne via global title crosswalk
if top_ne.empty:
    gg = df_global.copy()
    gg["lang_bucket"] = gg["category"].astype(str).str.contains(r"non-?
 ↪english", case=False, na=False)
    ne_keys = set(
        gg.loc[gg["lang_bucket"], ["show_title", "season_title"]]
          .dropna(subset=["show_title"])
          .itertuples(index=False, name=None)
    )
    cc["is_ne_by_title"] = cc.apply(
        lambda r: (r.get("show_title", None), r.get("season_title", None)) in␣
 ↪ne_keys, axis=1
    )
    top_ne = (
        cc.query("weekly_rank >= 1 and weekly_rank <= 10 and is_ne_by_title ==␣
 ↪True")
          .groupby("country_name", as_index=False)["weekly_rank"]
          .count()
          .rename(columns={"weekly_rank": "ne_top10_entries"})
          .sort_values("ne_top10_entries", ascending=False)
    )

# plot and create aesthetic visualizations
if top_ne.empty:
    print("Visual 2: No Non-English placements found in Countries data using␣
 ↪either method.")
else:
    top10 = top_ne.head(10).iloc[::-1]
    plt.style.use("dark_background")
    fig = plt.figure(figsize=(9, 10), dpi=120)
    ax = plt.gca()
```

```python
    accent = "#E50914"

    # grid only on x for readability
    ax.grid(True, axis="x", alpha=0.20, linewidth=0.8)
    ax.grid(False, axis="y")

    # bars with a soft glow
    ax.barh(top10["country_name"], top10["ne_top10_entries"],
            color=accent, alpha=0.90)

    # outline/"glow" pass
    for lw, alpha in [(10, 0.10), (6, 0.14)]:
        ax.barh(top10["country_name"], top10["ne_top10_entries"],
                color=accent, alpha=alpha)

    # title & axes
    ax.set_title("Top Countries by Non-English Top-10 Placements", fontsize=18,␣
↪pad=10, weight="bold")
    ax.set_xlabel("Non-English Top-10 Placements", fontsize=12)
    ax.set_ylabel("Country", fontsize=12)

    # hide spines
    for spine in ["top", "right", "left", "bottom"]:
        ax.spines[spine].set_visible(False)

    # value labels at bar ends
    for y, v in zip(top10["country_name"], top10["ne_top10_entries"]):
        ax.text(v + max(0.5, 0.01 * v), y, f"{int(v)}",
                va="center", ha="left", fontsize=11, color="white")

    plt.tight_layout()
    out_path = "visual2.png"
    plt.savefig(out_path, bbox_inches="tight")
    plt.show()
    print(f"Saved social version to: {out_path}")
```



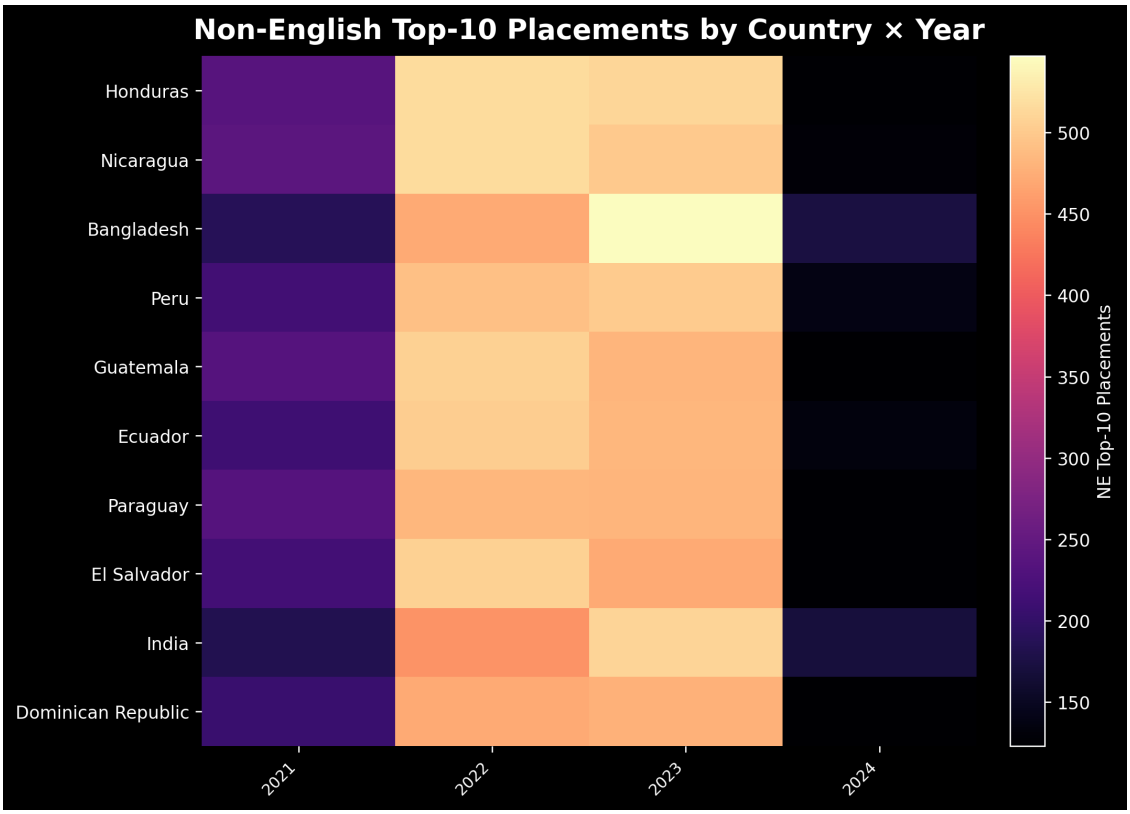

```
Saved social version to: visual2.png
```

0.0.3 Visual 3

[28]:
```
# country × year heatmap

# make copy and drop rows missing week or country_name
cc = df_countries.dropna(subset=["week", "country_name"]).copy()
# flag whether each record is ne
```

```
cc["is_ne"] = cc["category"].astype(str).str.contains(r"non-?english",␣
↪case=False, na=False)
# if direct flag finds nothing, crosswalk titles from global
needs_fallback = (cc["is_ne"].sum() == 0)
if needs_fallback:
    gg = df_global.copy()
    gg["is_ne_global"] = gg["category"].astype(str).str.contains(r"non-?
↪english", case=False, na=False)
    # create set of (show_title, season_title) keys that are ne in global
    ne_keys = set(
        gg.loc[gg["is_ne_global"], ["show_title", "season_title"]]
          .dropna(subset=["show_title"])
          .itertuples(index=False, name=None)
    )
    # flag rows in countries if combo matches that set
    cc["is_ne"] = cc.apply(
        lambda r: (r.get("show_title", None), r.get("season_title", None)) in␣
↪ne_keys, axis=1
    )

# aggregate: ne placements by (country, year) using top 10 ranks
cc["year"] = cc["week"].dt.year
heat = (
    cc.query("is_ne == True and weekly_rank >= 1 and weekly_rank <= 10")
      .groupby(["country_name", "year"], as_index=False)["weekly_rank"]
      .count()
      .rename(columns={"weekly_rank": "ne_placements"})
)

# safety check
if heat.empty:
    print("Visual 3: No Non-English placements found for heatmap (check tagging␣
↪or date ranges).")
else:
    # choose top 10 countries by total ne placements for readability
    top10_countries = (
        heat.groupby("country_name")["ne_placements"].sum()
            .sort_values(ascending=False).head(10).index.tolist()
    )
    heat10 = heat[heat["country_name"].isin(top10_countries)]

    # if no rows remain, report
    if heat10.empty:
        print("Visual 3: No data remained after selecting top countries.")
    else:
        years = sorted(heat10["year"].dropna().unique().tolist())
        countries_ord = top10_countries  # keep leaderboard order
```

```
    # build matrix (rows = countries, cols = years)
    mat = np.zeros((len(countries_ord), len(years)), dtype=float)
    for i, country in enumerate(countries_ord):
        row = (heat10[heat10["country_name"] == country]
               .set_index("year")["ne_placements"])
        for j, yr in enumerate(years):
            mat[i, j] = float(row.get(yr, 0))

    # plot
    plt.style.use("dark_background")
    fig = plt.figure(figsize=(9, 6.5), dpi=240)
    ax = plt.gca()

    # use perceptually uniform colormap
    im = ax.imshow(mat, aspect="auto", cmap="magma")

    # colorbar with label
    cbar = plt.colorbar(im, fraction=0.046, pad=0.04)
    cbar.set_label("NE Top-10 Placements", rotation=90)

    # ticks & labels
    ax.set_yticks(range(len(countries_ord)))
    ax.set_yticklabels(countries_ord, fontsize=10)
    ax.set_xticks(range(len(years)))
    ax.set_xticklabels(years, rotation=45, ha="right", fontsize=9)

    # title & cleanup
    ax.set_title("Non-English Top-10 Placements by Country × Year",␣
↪fontsize=16, pad=10, weight="bold")
    for spine in ["top", "right", "left", "bottom"]:
        ax.spines[spine].set_visible(False)

    plt.tight_layout()
    out_path = "visual3.png"
    plt.savefig(out_path, bbox_inches="tight")
    plt.show()
    print(f"Saved social version to: {out_path}")
```

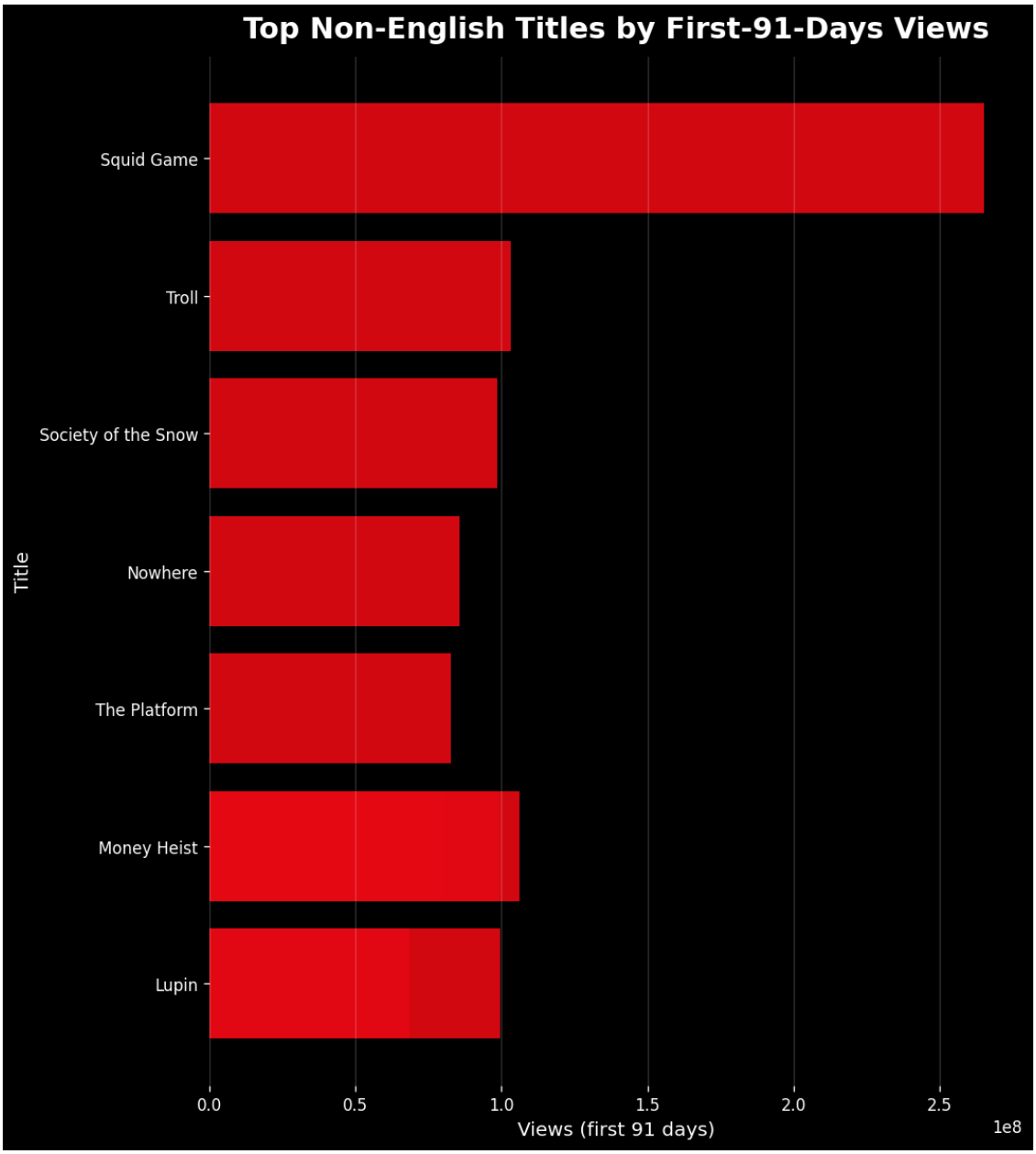



```
Saved social version to: visual3.png
```

0.0.4 Visual 4

[31]:
```
# most popular non-english titles (first 91 days views)

# make copy and language bucket
pop = df_pop.copy()
pop["lang_bucket"] = pop["category"].astype(str).str.contains(r"non-?english",␣
↪case=False, na=False)\
                     .map({True: "Non-English", False: "English"})

# ensure required columns exist
views_col = "views_first_91_days" if "views_first_91_days" in pop.columns else␣
↪None
title_col = "show_title" if "show_title" in pop.columns else None

if (views_col is None) or (title_col is None):
    print("Visual 4: Missing required columns (show_title /␣
↪views_first_91_days).")
else:
    # convert views to numeric + rank top non-english titles
```

```python
pop[views_col] = pd.to_numeric(pop[views_col], errors="coerce")
top_titles = (
    pop.loc[(pop["lang_bucket"] == "Non-English") & pop[views_col].notna()]
       .sort_values(views_col, ascending=False)
       .head(10)
       [[title_col, views_col]]
)

if top_titles.empty:
    print("Visual 4: No Non-English titles with 91-day views found.")
else:
    # plot
    plt.style.use("dark_background")
    fig = plt.figure(figsize=(9, 10), dpi=120)
    ax = plt.gca()

    accent = "#E50914"
    accent_dim = "#FF6B73"

    # horizontal bars
    bars = ax.barh(top_titles[title_col].iloc[::-1],
                   top_titles[views_col].iloc[::-1],
                   color=accent, alpha=0.9)

    # add faint glow behind bars
    for lw, alpha in [(10, 0.10), (6, 0.14)]:
        ax.barh(top_titles[title_col].iloc[::-1],
                top_titles[views_col].iloc[::-1],
                color=accent, alpha=alpha)

    # labels & axes
    ax.set_title("Top Non-English Titles by First-91-Days Views",
                 fontsize=18, pad=12, weight="bold")
    ax.set_xlabel("Views (first 91 days)", fontsize=12)
    ax.set_ylabel("Title", fontsize=12)

    # clean up axes
    ax.grid(True, axis="x", alpha=0.2, linewidth=0.8)
    ax.grid(False, axis="y")
    for spine in ["top", "right", "left", "bottom"]:
        ax.spines[spine].set_visible(False)

    plt.tight_layout()
    out_path = "visual4.png"
    plt.savefig(out_path, bbox_inches="tight")
    plt.show()
    print(f"Saved social version to: {out_path}")
```



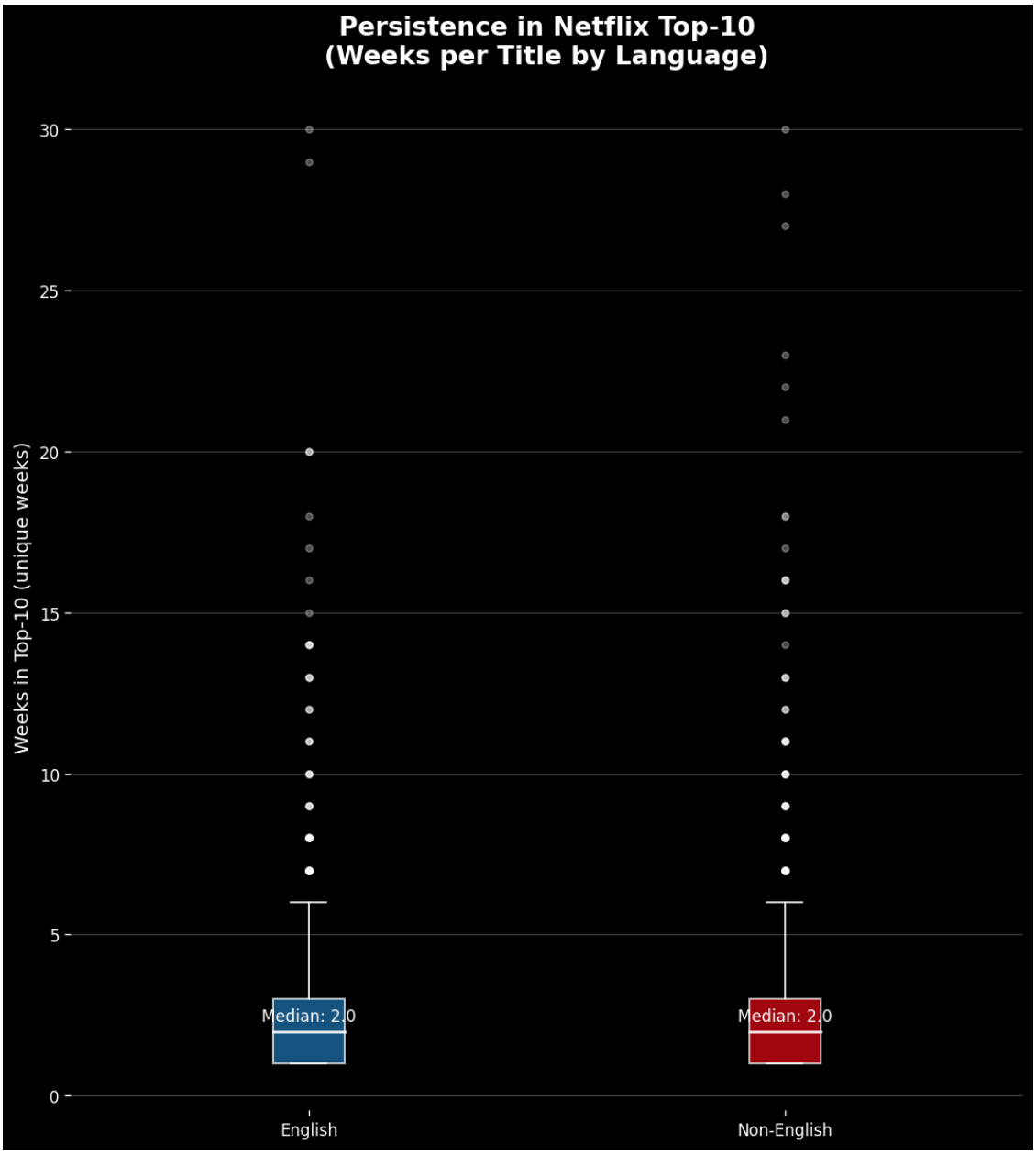

```
Saved social version to: visual4.png
```

0.0.5 Visual 5

[34]:
```
# persistence in top-10: english vs non-english

# make copy
gg5 = df_global.copy()

# validate required columns
```

```python
if "week" not in gg5.columns:
    print("Visual 5: Global file needs 'week' parsed to datetime.")
else:
    # add language bucket if not already created
    if "lang_bucket" not in gg5.columns:
        gg5["lang_bucket"] = gg5["category"].apply(lang_bucket)

    title_col = "show_title" if "show_title" in gg5.columns else None
    if title_col is None:
        print("Visual 5: Missing 'show_title' in global file.")
    else:
        # persistence = # of unique weeks title appeared in top-10
        pers = (
            gg5.dropna(subset=["week", title_col])
               .groupby([title_col, "lang_bucket"])["week"]
               .nunique()
               .reset_index(name="weeks_in_top10")
        )

        # split english vs non-english distributions
        ne_weeks = pers.loc[pers["lang_bucket"] == "Non-English",␣
↪"weeks_in_top10"].values
        en_weeks = pers.loc[pers["lang_bucket"] == "English", "weeks_in_top10"].
↪values

        if (len(ne_weeks) == 0) or (len(en_weeks) == 0):
            print("Visual 5: Not enough data to compare persistence by language.
↪")
        else:
            # plot
            plt.style.use("dark_background")
            fig = plt.figure(figsize=(9, 10), dpi=120)
            ax = plt.gca()

            accent_en = "#1f77b4"
            accent_ne = "#E50914"

            # boxplot
            box = ax.boxplot([en_weeks, ne_weeks],
                             labels=["English", "Non-English"],
                             patch_artist=True,
                             medianprops=dict(color="white", linewidth=1.5),
                             boxprops=dict(linewidth=1.2, color="white"),
                             whiskerprops=dict(color="white"),
                             capprops=dict(color="white"),
                             flierprops=dict(markerfacecolor="white",␣
↪marker="o", markersize=4, alpha=0.3))
```

```
        # color boxes
        colors = [accent_en, accent_ne]
        for patch, color in zip(box["boxes"], colors):
            patch.set_facecolor(color)
            patch.set_alpha(0.7)

        # titles & axes
        ax.set_title("Persistence in Netflix Top-10\n(Weeks per Title by␣
↪Language)",
                     fontsize=16, pad=12, weight="bold")
        ax.set_ylabel("Weeks in Top-10 (unique weeks)", fontsize=12)

        # subtle grid for readability
        ax.grid(True, axis="y", alpha=0.25, linewidth=0.8)
        ax.grid(False, axis="x")

        # clean spines
        for spine in ["top", "right", "left", "bottom"]:
            ax.spines[spine].set_visible(False)

        # show medians on chart
        medians = [np.median(en_weeks), np.median(ne_weeks)]
        for i, med in enumerate(medians, start=1):
            ax.text(i, med + 0.2, f"Median: {med:.1f}",
                    ha="center", va="bottom", fontsize=10, color="white")

        plt.tight_layout()
        out_path = "visual5.png"
        plt.savefig(out_path, bbox_inches="tight")
        plt.show()
        print(f"Saved social version to: {out_path}")
```




```
Saved social version to: visual5.png
```

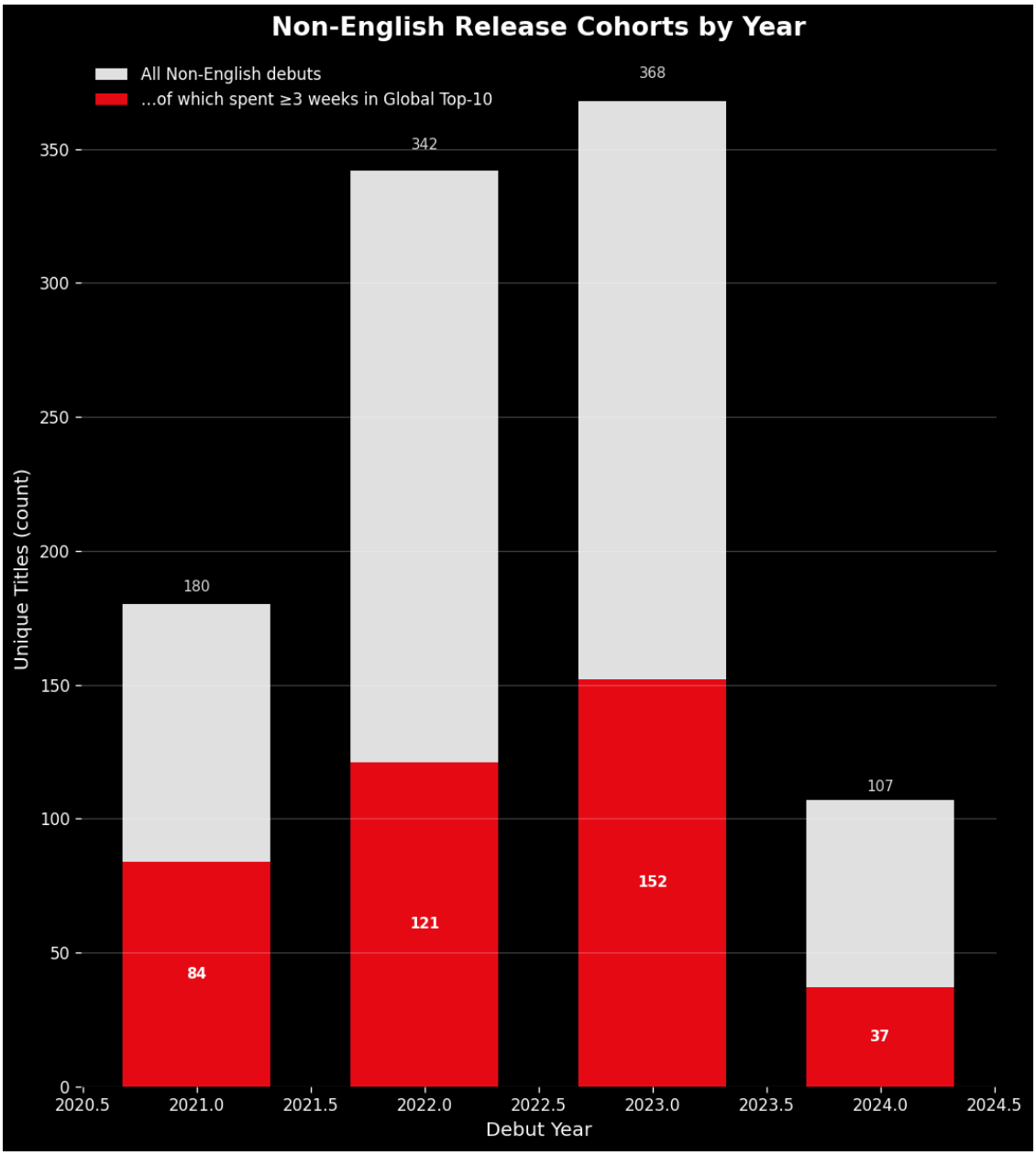
0.0.6 Visual 6

[37]:
```
# release cohorts by year
# full bar = all non-english debuts
# red overlay = titles that spend 3 or more weeks in global top-10

def _safe_title_key(df):
```

```
    """stable title key: (show_title • season_title) if season exists; else␣
↪show_title (lowercased, trimmed)"""
    if "show_title" not in df.columns:
        raise ValueError("Expected column 'show_title'.")
    show = df["show_title"].astype(str).str.strip().str.lower()
    if "season_title" in df.columns:
        season = df["season_title"].astype(str).fillna("").str.strip().str.
↪lower()
        return show + " • " + season
    return show

def _ensure_is_ne(df):
    """non-english flag derived from 'category' (handles 'Non-English' / 'Non␣
↪English')"""
    if "is_ne" in df.columns:
        return df["is_ne"].astype(bool)
    if "category" in df.columns:
        return df["category"].astype(str).str.contains(r"non[\s-]?english",␣
↪case=False, na=False)
    return pd.Series(False, index=df.index)

def visual6_release_cohorts_highlight_threshold(df_countries, df_global,
                                                min_weeks_global=3,
                                                min_year=None, max_year=None):
    cc = df_countries.copy()
    gg = df_global.copy()
    if "week" in cc.columns: cc["week"] = pd.to_datetime(cc["week"],␣
↪errors="coerce")
    if "week" in gg.columns: gg["week"] = pd.to_datetime(gg["week"],␣
↪errors="coerce")

    def _filter_top10(df):
        """apply top-10 filter only if it actually filters (rank col exists &␣
↪valid)"""
        for col in ("weekly_rank", "weekly rank"):
            if col in df.columns:
                m = pd.to_numeric(df[col], errors="coerce").between(1, 10)
                if m.sum() > 0:
                    return df[m].copy()
        return df
    cc = _filter_top10(cc)
    gg = _filter_top10(gg)

    # non-english rows + debut year
    cc_ne = _ensure_is_ne(cc)
    gg_ne = _ensure_is_ne(gg)
```

```
    cc_ne_rows = cc[cc_ne & cc["week"].notna()].copy()
    gg_ne_rows = gg[gg_ne & gg["week"].notna()].copy()
    if cc_ne_rows.empty and gg_ne_rows.empty:
        print("Visual 6: No Non-English rows with valid weeks in Countries/
↪Global.")
        return

    cc_ne_rows["_key"] = _safe_title_key(cc_ne_rows)
    gg_ne_rows["_key"] = _safe_title_key(gg_ne_rows)

    # combine countries & global to find each title's first week (debut)
    comb = pd.concat([cc_ne_rows[["week","_key"]],␣
↪gg_ne_rows[["week","_key"]]], ignore_index=True)
    debut = (comb.groupby("_key")["week"].min()
                  .rename("debut_week").reset_index()
                  .assign(debut_year=lambda d: d["debut_week"].dt.year))

    # identify cross-overs: titles with at least  min_weeks_global in global␣
↪top-10
    if "cumulative_weeks_in_top_10" in gg_ne_rows.columns:
        gg_ne_rows["cumulative_weeks_in_top_10"] = pd.to_numeric(
            gg_ne_rows["cumulative_weeks_in_top_10"], errors="coerce"
        )
        global_weeks = (gg_ne_rows.groupby("_key")["cumulative_weeks_in_top_10"]
                                  .max().rename("weeks_global"))
    else:
        global_weeks = (gg_ne_rows.groupby("_key")["week"]
                                  .nunique().rename("weeks_global"))

    crossover_titles = set(global_weeks[global_weeks >= min_weeks_global].index)
    debut["is_crossover"] = debut["_key"].isin(crossover_titles)

    # optional year filter
    if min_year is not None: debut = debut[debut["debut_year"] >= int(min_year)]
    if max_year is not None: debut = debut[debut["debut_year"] <= int(max_year)]
    if debut.empty:
        print("Visual 6: No debuts after year filter(s).")
        return

    # aggreate per debut year: totals + cross-overs
    yearly = (debut.groupby("debut_year")["is_crossover"]
                   .agg(total="count", cross="sum").reset_index())
    years = yearly["debut_year"].astype(int).values
    total = yearly["total"].values
    cross = yearly["cross"].values
```

```
    # plot
    plt.style.use("dark_background")
    fig = plt.figure(figsize=(9, 10), dpi=120)
    ax = plt.gca()

    accent = "#E50914"
    ax.grid(True, axis="y", alpha=0.25, linewidth=0.8)
    ax.grid(False, axis="x")

    # base bar = all ne debuts
    ax.bar(years, total, width=0.65, color="#E0E0E0", label="All Non-English␣
↪debuts")

    # overlay = cross-overs meeting threshold
    ax.bar(years, cross, width=0.65, color=accent,
           label=f"…of which spent  {min_weeks_global} weeks in Global Top-10")

    # labels & axes
    ax.set_title("Non-English Release Cohorts by Year", fontsize=16, pad=10,␣
↪weight="bold")
    ax.set_xlabel("Debut Year", fontsize=12)
    ax.set_ylabel("Unique Titles (count)", fontsize=12)
    for spine in ["top", "right", "left", "bottom"]:
        ax.spines[spine].set_visible(False)

    # annotations
    for x, t, c in zip(years, total, cross):
        if t > 0:
            ax.text(x, t + max(0.5, 0.02*t), f"{int(t)}",
                    ha="center", va="bottom", fontsize=9, color="#DDDDDD")
        if c > 0:
            ax.text(x, c/2, f"{int(c)}",
                    ha="center", va="center", fontsize=9, color="white",␣
↪fontweight="bold")

    ax.legend(frameon=False, loc="upper left")
    plt.tight_layout()

    out_path = "visual6.png"
    plt.savefig(out_path, bbox_inches = "tight")
    plt.show()
    print(f"Saved social version to: {out_path}")

# run
visual6_release_cohorts_highlight_threshold(df_countries, df_global)
```




```
Saved social version to: visual6.png
```

The intended audience of this project is the general public and social media users, focusing on those interested in entertainment trends. Using clear and bold visuals along with simple storytelling and explanations, the goal was to create content that can be used in short-form (i.e. Instagram or TikTok) to deliver impact quickly. The purpose of this story was to highlight the rise in popularity and global impact of Non-English Netflix titles. It recognizes that audiences worldwide are embracing Non-English productions, in hopes that the audience walks away knowing that the trend is not temporary; it's growing and showing strong persistence. The selected medium for the project was short-form social media infographics designed as a carousel of 8 slides (6 visuals in total) and best

matches with the target audience. It includes a short title and one-sentence sub-tagline to fit in with the narrative. The design choices took from the current Netflix branding, utilizing a dark/black background, with white and red to emphasize key points. Reducing spines, adding subtle grid lines, and reducing clutter makes it easy to consume, even on a mobile device, without overwhelming the audience. The changes made to the data include cleaning by parsing dates, converting numeric fields, removing duplicates, and flagging Non-English titles consistently. Filters were also applies such as Top 10 for readability. The datasets come directly from Netflix's publicly released Global Top 10 lists and does not include any private or personally identifiable information (PII). Misinterpretation is possible if the audience assumes Non-English titles dominate all viewing, but was mitigated by framing results as an going story, not a replacement of English shows/movies. Assumptions were limited to classifying Non-English content using the category field, but were labelled within the code and visuals. All data was published by Netflix and is publically accesible, ensuring credibility and ethical acquisition. To mitigate any ethical implications, transparency is important which includes providing descriptive titles, labels, and notes about thresholds.

[ ]:

```

```